

Cahier des charges : Mise en œuvre d'un système de pointage

1. Introduction

Le projet consiste à concevoir et développer un système de pointage moderne permettant de gérer les présences, absences, et retards au sein de l'entreprise. Ce système utilisera Django Rest Framework (backend) pour les API et Next.js (frontend) pour une interface utilisateur réactive et ergonomique.

2. Objectifs

- **Général** : Automatiser et simplifier le suivi des présences.
- **Spécifiques** :
 1. Créer une API REST sécurisée et performante.
 2. Concevoir une interface utilisateur intuitive pour les employés et les administrateurs.
 3. Fournir des rapports et statistiques détaillés sur la gestion du temps.
 4. Gérer les utilisateurs, les rôles, et les autorisations.

3. Fonctionnalités principales

Backend (Django Rest Framework) :

1. **Gestion des utilisateurs et rôles** :
 - Création, mise à jour, suppression des comptes utilisateurs.
 - Rôles : manager, employé.
2. **Pointage** :
 - Enregistrement des heures d'arrivée et de départ.
 - Marquage automatique des retards.
3. **Gestion des absences** :
 - Demande, validation, et historique des absences.
4. **Rapports et statistiques** :
 - Filtrage par période
 - Visualisation des statistiques.
5. **Sécurité** :
 - Authentification avec JWT.
 - Protection des données sensibles.

Frontend (Next.js) :

1. **Interface utilisateur** :
 - Tableau de bord pour les employés : pointage et historique.

- Tableau de bord administrateur : suivi global, validation des absences.
- 2. **Notifications :**
 - Notifications en temps réel pour les retards et demandes d'absence.
- 3. **Rapports interactifs :**
 - Graphiques dynamiques pour les statistiques.
 - Options de téléchargement des rapports.

4. Architecture technique

1. **Backend :**
 - Framework : Django + DRF.
 - Base de données : PostgreSQL.
 - Authentification : JWT.
2. **Frontend :**
 - Framework : Next.js. UI : Tailwind CSS.
3. **Communication Backend-Frontend :**
 - API REST avec des endpoints documentés via Swagger.
4. **Déploiement :**
 - Conteneurisation : Docker et Docker compose.

5. Contraintes

- Adaptation mobile (responsive design).

6. Livrables

1. Code source (frontend, backend).
2. Documentation et diagramme UML
3. Dockerfile et docker-compose.yaml pour déploiement.

7. Période de réalisation

Durée estimée : 1 mois (deux semaines pour backend et deux semaines pour frontend)

8. Instructions :

Backend

- Analyser le projet et produire un diagramme de classe.
- Découper le projet en petites applications (app pour auth, app pour gestion de pointage, app pour gestion d'absence, app pour statistiques, etc.).
- Commenter votre code (les docstrings sont vos amis).
- Utiliser les `ModelViewSet`, les actions et les permissions personnalisées.

Frontend

- Lors de la création du projet Next.js, utiliser JavaScript au lieu de TypeScript.
- Choisir le routage basé sur le système de fichiers.
- Créer autant de composants React que possible.